

Python开发工程师题库

——从入门到精通职业技能培训配套题库

题库说明

1. 本题库共23章，每章12道题，总计276道题。
2. 题型包括单选题、多选题和判断题，覆盖Python从入门到精通全部知识点。
3. 每道题后紧跟答案和解析，方便系统解析和学员自测。
4. 题目紧扣职业场景，注重实际应用能力考核。

第1章 Python简介与环境搭建

1. 【单选题】Python属于以下哪种语言类型？

- A. 编译型语言
- B. 解释型语言
- C. 汇编语言
- D. 机器语言

答案：B

解析：Python是解释型语言，代码由Python解释器逐行执行，无需预先编译成机器码。

2. 【单选题】Python 3中， $10 / 4$ 的结果是？

- A. 2
- B. 2.5
- C. 2
- D. 报错

答案：B

解析：Python 3中/执行真除法，返回浮点数2.5；//才执行整除。

3. 【单选题】以下哪个不是Python的主要特点？

- A. 动态类型
- B. 强制缩进
- C. 编译执行
- D. 面向对象

答案：C

解析：Python是解释执行而非编译执行（虽然有.pyc字节码缓存，但对用户透明）。

4. 【单选题】Python 2.x已于哪一年停止维护？

- A. 2018年
- B. 2019年
- C. 2020年
- D. 2021年

答案：C

解析：Python 2.x已于2020年1月1日停止维护，生产环境必须使用Python 3.x。

5. 【单选题】企业开发建议选择的Python最低版本是？

- A. Python 3.5
- B. Python 3.6
- C. Python 3.8
- D. Python 3.10

答案：C

解析：Python 3.8+支持赋值表达式（海象运算符），且稳定性好，是企业开发的推荐最低版本。

6. 【多选题】Python的主要特点包括哪些？

- A. 简洁优雅
- B. 解释型语言
- C. 动态类型
- D. 跨平台

答案：ABCD

解析：Python具有简洁优雅的语法、解释执行、动态类型、跨平台等全部特点。

7. 【多选题】Python开发环境配置中，以下哪些工具是推荐的？

- A. PyCharm
- B. VS Code
- C. Jupyter Notebook
- D. IDLE

答案：ABC

解析：PyCharm、VS Code、Jupyter Notebook都是主流开发工具，IDLE功能简单不适合职业开发。

8. 【多选题】以下哪些操作属于Python环境搭建的正确步骤？

- A. 从python.org下载安装包
- B. 勾选Add Python to PATH
- C. 使用venv创建虚拟环境
- D. 在全局环境中直接安装所有依赖

答案：ABC

解析：应避免在全局环境中直接安装依赖，应使用虚拟环境隔离项目依赖。

9. 【判断题】Python 2.x和Python 3.x完全兼容，代码可以无缝迁移。

答案：错误

解析：Python 2和3不完全兼容，例如print语句变为函数，unicode和str合并等，迁移需要修改代码。

10. 【判断题】Python使用缩进来表示代码块，而不是花括号。

答案：正确

解析：Python强制使用缩进（通常为4个空格）来标识代码块，这是其语法特色。

11. 【判断题】Windows上安装Python后，必须手动配置环境变量才能使用python命令。

答案：错误

解析：安装时勾选Add Python to PATH即可自动配置，无需手动设置。

12. 【判断题】Jupyter Notebook适合探索性数据分析，但不适合大型项目工程化开发。

答案：正确

解析：Jupyter适合数据探索和原型验证，大型项目应使用PyCharm或VS Code进行工程化管理。

第2章 Python基础语法

13. 【单选题】以下哪个变量名是合法的？

- A. 2name
- B. _private
- C. class
- D. my-name

答案：B

解析：A以数字开头非法；C是关键字；D含连字符非法（会被解释为减号）。

14. 【单选题】执行`3 * 'abc'`的结果是？

- A. 9
- B. 'abccabccabc'
- C. 报错
- D. '3abc'

答案：B

解析：字符串乘整数表示重复，`'abc' * 3`得到'abccabccabc'。

15. 【单选题】以下代码的输出是？`a = 5; b = 2; print(a // b, a / b, a % b)`

- A. 2 2.5 1
- B. 2.5 2.5 1
- C. 2 2 1
- D. 2 2.5 0

答案：A

解析：`//`整除得2，`/`真除法得2.5，`%`取余得1。

16. 【单选题】`f"{3.14159:.2f}"`的结果是？

- A. '3.14'
- B. '3.14159'
- C. '3.15'
- D. '3.1'

答案：A

解析：`:.2f`表示保留两位小数，四舍五入后3.14。

17. 【单选题】`type(5/2)`的结果是？

- A.
- B.
- C.
- D.

答案：B

解析：5/2返回2.5，类型是float；5//2才是int。

18. 【多选题】以下哪些是Python的基本数据类型？

- A. int
- B. float
- C. str
- D. char

答案：ABC

解析：Python基本类型包括int、float、bool、str、NoneType，没有char类型。

19. 【多选题】以下关于字符串的说法正确的是？

- A. 字符串是不可变对象
- B. s[0] = 'A'会报错
- C. 字符串乘整数表示重复
- D. join()方法用于分割字符串

答案：ABC

解析：join()用于连接字符串而非分割，split()才是分割方法。

20. 【多选题】以下哪些是合法的变量命名？

- A. user_name
- B. _count
- C. MAX_SIZE
- D. 1st_place

答案：ABC

解析：变量名不能以数字开头，1st_place不合法。

21. 【判断题】Python中字符串是可变对象，可以直接修改某个字符。

答案：错误

解析：字符串是不可变对象，s[0] = 'A'会报错，需通过切片或replace()创建新字符串。

22. 【判断题】大量字符串拼接时，使用join()方法比使用+操作符更高效。

答案：正确

解析：join()预先分配足够内存，只创建一次新字符串；+拼接会创建大量中间字符串对象。

23. 【判断题】Python中0、空字符串、空列表、None在条件判断中都被视为False。

答案：正确

解析：Python中0、0.0、''、[]、{}、None、False等为假值，其余为真值。

24. 【判断题】金融计算中应使用float类型表示金额，因为float精度足够。

答案：错误

解析：float存在精度问题（如 $0.1+0.2\neq 0.3$ ），金融计算应使用decimal模块的Decimal类型。

第3章 流程控制

25. 【单选题】以下代码的输出是？`for i in range(3): if i == 1: continue; print(i, end='')`

- A. 012
- B. 02
- C. 13
- D. 01

答案：B

解析：i==1时执行continue跳过打印，所以只输出0和2。

26. 【单选题】`range(5, 0, -1)`生成的序列是？

- A. [5, 4, 3, 2, 1, 0]
- B. [5, 4, 3, 2, 1]
- C. [5, 3, 1]
- D. 空序列

答案：B

解析：`range(5, 0, -1)`从5递减到1（左闭右开，不包含0）。

27. 【单选题】以下关于while-else的说法正确的是？

- A. else总是执行
- B. else仅在循环被break时执行
- C. else仅在循环未被break时执行
- D. while不能有else

答案：C

解析：while-else和for-else的else仅在循环未被break中断时执行。

28. 【单选题】以下代码的输出是？`x = 5; if x > 10: print('A'); elif x > 3: print('B'); elif x > 1: print('C')`

- A. A
- B. B
- C. C
- D. BC

答案：B

解析：`x > 3`为真，执行`print('B')`，elif条件满足后不再检查后续条件。

29. 【单选题】三元表达式`a if condition else b`中，当condition为True时返回？

- A. a
- B. b
- C. condition

D. None

答案：A

解析：三元表达式在condition为True时返回a，否则返回b。

30. 【多选题】以下哪些是Python中的循环控制关键字？

- A. break
- B. continue
- C. return
- D. pass

答案：ABC

解析：break退出循环，continue跳过当前迭代，return退出函数（也会退出循环），pass是空语句不控制循环。

31. 【多选题】以下关于range的说法正确的是？

- A. range(10)生成0到9
- B. range(1, 10, 2)生成1,3,5,7,9
- C. range(10, 0, 2)生成空序列
- D. range是列表类型

答案：ABC

解析：range返回range对象（可迭代），不是列表，但可转换为list。

32. 【多选题】以下哪些说法是正确的？

- A. if条件可以是任意表达式
- B. 非零、非空即为真
- C. for循环可以遍历字典的items()
- D. while循环必须有else子句

答案：ABC

解析：while和for的else子句是可选的，不是必须的。

33. 【判断题】if语句中，条件可以是任意表达式，非零、非空即为真。

答案：正确

解析：Python中0、0.0、''、[]、{}、None、False等为假，其余为真。

34. 【判断题】for i in range(10, 0, 2)会生成一个空序列。

答案：正确

解析：起始值10已经大于终止值0，且步长为正方向，因此生成空序列。

35. 【判断题】在遍历列表时可以直接删除列表元素，不会影响遍历结果。

答案：错误

解析：在遍历列表时修改列表（增删元素）会导致索引错乱，应创建新列表或使用列表推导式。

36. 【判断题】Python 3.10+支持match-case模式匹配，可以替代复杂的if-elif链。

答案：正确

解析：match-case支持模式匹配，适合替代复杂的多分支条件判断。

第4章 数据结构

37. 【单选题】以下哪个不能作为字典的键？

- A. 'name'
- B. 42
- C. (1, 2)
- D. [1, 2]

答案：D

解析：字典键必须是不可变（可哈希）对象，列表是可变的，不能作为键。

38. 【单选题】执行list('abc')的结果是？

- A. ['abc']
- B. ['a', 'b', 'c']
- C. 报错
- D. [97, 98, 99]

答案：B

解析：list()将可迭代对象转为列表，字符串迭代得到单个字符。

39. 【单选题】以下代码的输出是？s = {1, 2, 3}; s.add(3); print(len(s))

- A. 3
- B. 4
- C. 2
- D. 报错

答案：A

解析：集合自动去重，add(3)时3已存在，长度仍为3。

40. 【单选题】列表推导式[x for x in range(10) if x % 2 == 0 if x % 3 == 0]的结果是？

- A. [0, 6]
- B. [0, 2, 4, 6, 8]
- C. [0, 3, 6, 9]
- D. [6]

答案：A

解析：同时满足被2整除和被3整除，即被6整除，0~9中只有0和6。

41. 【单选题】dict.get(key)在键不存在时返回什么？

- A. KeyError

- B. None
- C. 0
- D. 空字符串

答案：B

解析：dict.get(key, default)安全访问，键不存在时返回默认值（默认为None）。

42. 【多选题】以下关于列表的说法正确的是？

- A. 列表是有序的
- B. 列表是可变的
- C. 列表可以包含重复元素
- D. 列表索引时间复杂度是O(1)

答案：ABCD

解析：列表是有序、可变、可重复的序列，索引访问时间复杂度为O(1)。

43. 【多选题】以下哪些操作可以复制列表？

- A. `b = a`
- B. `b = a[:]`,
- C. `b = list(a)`
- D. `b = copy.deepcopy(a)`

答案：BCD

解析：`b = a`只是增加引用，不会复制列表。切片`[:]`、`list()`、`deepcopy()`都能创建新列表。

44. 【多选题】以下关于集合的说法正确的是？

- A. 集合是无序的
- B. 集合元素不重复
- C. 空集合用`{}`表示
- D. 集合支持交集运算

答案：ABD

解析：空集合必须用`set()`，`{}`会被识别为空字典。

45. 【判断题】元组是不可变的，因此元组中的列表元素也不能被修改。

答案：错误

解析：元组本身不可变，但如果元素是可变对象（如列表），其内容可以修改。

46. 【判断题】列表作为函数默认参数会导致状态共享，应使用None作为默认值。

答案：正确

解析：默认参数在函数定义时求值一次，可变对象（列表、字典）会在多次调用间共享状态。

47. 【判断题】字典推导式`{x: x**2 for x in range(5)}`会生成`{0: 0, 1: 1, 2: 4, 3: 9, 4: 16}`。

答案：正确

解析：字典推导式的语法正确，会生成指定键值对的字典。

48. 【判断题】Python 3.7+字典保持插入顺序，因此可以依赖字典的顺序进行位置相关操作。

答案：错误

解析：虽然3.7+字典保持插入顺序，但不应依赖此特性进行位置相关操作，应使用列表或collections.Ordered Dict。

第5章 函数基础

49. 【单选题】以下关于函数默认参数的说法，正确的是？

- A. 默认参数必须是最后一个参数
- B. 默认参数在每次调用时都会重新计算默认值
- C. 默认参数如果是可变对象，可能在多次调用间共享状态
- D. 默认参数不能用关键字方式传入

答案：C

解析：默认参数在函数定义时求值一次，如果是列表、字典等可变对象，多次调用会共享同一个对象。

50. 【单选题】执行以下代码的结果是？`def func(a, b, c=3): return a + b + c; print(func(1, c=5, b=2))`

- A. 6
- B. 8
- C. 报错
- D. 11

答案：B

解析：关键字参数c=5, b=2可以打乱顺序，计算1 + 2 + 5 = 8。

51. 【单选题】*args在函数内部的数据类型是？

- A. 列表
- B. 元组
- C. 字典
- D. 集合

答案：B

解析：*args收集多余位置参数，内部是元组类型（不可变）。

52. 【单选题】以下代码的输出是？`x = 10; def foo(): x = 20; def bar(): nonlocal x; x = 30; bar(); return x; print(foo())`

- A. 10
- B. 20
- C. 30
- D. 报错

答案：C

解析：nonlocal声明x来自外层（非全局）作用域，bar()中修改的是foo()中的x，最终返回30。

53. 【单选题】lambda表达式可以包含什么？

- A. 任意复杂的语句
- B. 多条代码
- C. 单个表达式
- D. 循环语句

答案：C

解析：lambda只能包含单个表达式，不能有多条语句、赋值、循环等复杂逻辑。

54. 【多选题】以下关于Python函数的说法正确的是？

- A. 函数可以返回多个值
- B. 函数是一等公民
- C. lambda可以包含多条语句
- D. 函数可以嵌套定义

答案：ABD

解析：lambda只能包含单个表达式，不能有多条语句。函数可以返回多个值（实际是元组），是一等公民，支持嵌套。

55. 【多选题】以下哪些是合法的参数组合？

- A. def f(a, b, c=3)
- B. def f(a, *args, b=2)
- C. def f(a, b, *args, c, **kwargs)
- D. def f(*args, **kwargs)

答案：ABCD

解析：以上参数组合都是合法的，展示了位置参数、默认参数、可变参数、仅限关键字参数的灵活组合。

56. 【多选题】以下关于作用域的说法正确的是？

- A. global声明全局变量
- B. nonlocal声明外层非全局变量
- C. LEGB规则中L代表Local
- D. 函数内部可以直接修改全局变量

答案：ABC

解析：函数内部修改全局变量需先声明global，否则创建的是局部变量。

57. 【判断题】Python函数可以返回多个值，实际上返回的是一个元组。

答案：正确

解析：return a, b等价于return (a, b)，调用时可用x, y = func()解包。

58. 【判断题】函数默认参数使用可变对象（如列表）时，每次调用都会创建新的列表。

答案：错误

解析：默认参数在函数定义时求值一次，可变对象会在多次调用间共享同一个对象。

59. 【判断题】使用global关键字可以在函数内部修改全局变量的值。

答案：正确

解析：global声明后，函数内部对变量的修改会影响全局作用域中的变量。

60. 【判断题】nonlocal关键字可以用于修改全局变量的值。

答案：错误

解析：nonlocal用于修改外层（非全局）作用域的变量，修改全局变量应使用global。

第6章 文件操作

61. 【单选题】以下哪种方式打开文件，如果文件不存在会创建新文件？

- A. 'r'
- B. 'x'
- C. 'a'
- D. B和C

答案：D

解析：'x'独占创建（文件已存在会报错），'a'追加模式（不存在则创建），'w'也会创建但会清空已存在文件。

62. 【单选题】关于with open(...) as f:，以下说法错误的是？

- A. 文件会在with块结束时自动关闭
- B. 即使发生异常，文件也会被关闭
- C. 等同于f = open(...); f.close()
- D. 可以省略as f部分

答案：D

解析：as f将文件对象绑定到变量f，如果省略则无法在with块内操作文件。

63. 【单选题】pathlib.Path('a/b/c.txt').stem的值是？

- A. 'c.txt'
- B. 'c'
- C. '.txt'
- D. 'a/b/c'

答案：B

解析：.stem返回文件名去掉后缀的部分，'c.txt'的stem是'c'。

64. 【单选题】大文件处理时，推荐的方式是？

- A. f.read()
- B. f.readlines()
- C. for line in f:

D. `f.read(1024)`

答案：C

解析：for line in f 逐行迭代，内存中只保留一行，适合大文件。

65. 【单选题】以'w+'模式打开文件，以下说法正确的是？

- A. 不清空原有内容
- B. 支持读写
- C. 文件不存在会报错
- D. 只能写入不能读取

答案：B

解析：'w+'支持读写，但会清空文件内容（如果文件存在）；'r+'支持读写且不清空。

66. 【多选题】以下哪些是文件打开模式？

- A. 'r'
- B. 'w'
- C. 'a'
- D. 'x'

答案：ABCD

解析：r只读、w只写（清空）、a追加、x独占创建，都是标准打开模式。

67. 【多选题】以下关于CSV处理的说法正确的是？

- A. csv模块是Python标准库
- B. `csv.DictReader`返回字典形式的数据
- C. CSV文件用Excel打开中文乱码时应使用utf-8-sig编码
- D. CSV有类型信息，读取后自动转换为对应类型

答案：ABC

解析：CSV没有类型信息，所有字段读取后都是字符串，需手动转换。

68. 【多选题】以下关于JSON处理的说法正确的是？

- A. `json.loads`解析JSON字符串
- B. `json.dumps`生成JSON字符串
- C. `json.load`从文件读取JSON
- D. `json.dumps`设置`ensure_ascii=False`可输出中文

答案：ABCD

解析：以上都是json模块的正确用法。

69. 【判断题】在Windows上使用pathlib拼接路径时，需要手动使用反斜杠\。

答案：错误

解析：pathlib自动处理跨平台路径分隔符，使用/即可，在Windows上会自动转换为\。

70. 【判断题】以'w+'模式打开文件，既可以读也可以写，且不会清空原有内容。

答案：错误

解析：'w+'会清空文件内容（如果文件存在），'r+'才不会清空且支持读写。

71. 【判断题】使用with语句打开文件可以确保文件在发生异常时也能正确关闭。

答案：正确

解析：with语句通过上下文管理器确保文件在块结束时自动关闭，即使发生异常也会调用close()。

72. 【判断题】处理大JSON文件时，可以使用ijson库实现流式解析，避免内存溢出。

答案：正确

解析：ijson支持增量解析JSON，适合处理超大JSON文件，无需一次性载入内存。

第7章 异常处理

73. 【单选题】以下代码的输出是？
`try: print('A'); raise ValueError('test'); print('B'); except
ValueError: print('C'); finally: print('D')`

A. A B C D

B. A C D

C. A D C

D. A C

答案：B

解析：raise后print('B')不会执行；except捕获后打印C；finally总是执行打印D。

74. 【单选题】以下关于except:（不带异常类型）的说法，正确的是？

A. 只捕获Exception的子类

B. 捕获所有异常，包括KeyboardInterrupt

C. 性能比指定异常类型更好

D. 是推荐的做法

答案：B

解析：裸except:捕获BaseException下的所有异常，包括KeyboardInterrupt和SystemExit，会干扰程序正常退出，极不推荐。

75. 【单选题】raise RuntimeError('msg') from e的作用是？

A. 忽略原始异常e

B. 将e作为新异常的原因，保留异常链

C. 同时抛出两个异常

D. 捕获异常e

答案：B

解析：from e建立异常链，新异常的__cause__指向原始异常，便于追溯根因。

76. 【单选题】assert condition在什么情况下会抛出异常？

- A. condition为True
- B. condition为False
- C. 总是抛出
- D. 从不抛出

答案：B

解析：assert断言机制，条件为假时抛出AssertionError，用于调试检查前置条件。

77. 【单选题】自定义异常应该继承自哪个类？

- A. BaseException
- B. Exception
- C. Error
- D. RuntimeError

答案：B

解析：自定义异常应继承Exception（或其子类）。继承BaseException会捕获系统级异常，不符合常规错误语义。

78. 【多选题】以下关于异常处理的说法正确的是？

- A. finally块中的代码一定会执行
- B. 优先捕获具体异常类型
- C. 可以使用raise ... from e建立异常链
- D. 裸except:是推荐的做法

答案：ABC

解析：裸except:极不推荐，会捕获KeyboardInterrupt等系统级异常，干扰程序正常退出。

79. 【多选题】以下哪些是Python内置的异常类型？

- A. ValueError
- B. TypeError
- C. KeyError
- D. BusinessError

答案：ABC

解析：ValueError、TypeError、KeyError都是内置异常，BusinessError是自定义异常。

80. 【多选题】以下关于try-except-finally的说法正确的是？

- A. else子句在无异常时执行
- B. finally子句无论有无异常都执行
- C. finally中遇到return不会执行
- D. 可以没有except只有finally

答案：ABD

解析：finally块一定会执行，即使在try或except中遇到return，也是先执行finally再返回。

81. 【判断题】finally块中的代码在函数内遇到return时不会执行。

答案：错误

解析：finally块一定会执行，即使在try或except中遇到return，也是先执行finally再返回。

82. 【判断题】自定义异常应该继承自BaseException而不是Exception。

答案：错误

解析：自定义异常应继承Exception（或其子类）。继承BaseException会捕获系统级异常，不符合常规错误语义。

83. 【判断题】使用contextlib.suppress可以静默忽略指定的异常。

答案：正确

解析：with suppress(FileNotFoundError): os.remove('temp.txt')会在文件不存在时静默忽略删除错误。

84. 【判断题】assert语句在生产环境中会被自动移除，不能用于关键校验。

答案：正确

解析：assert在优化模式（-O）下会被移除，不能用于关键业务校验，应使用if + raise。

第8章 模块与包

85. 【单选题】以下关于if __name__ == '__main__':的说法，正确的是？

- A. 被导入时也会执行
- B. 只有直接运行该文件时才会执行
- C. 是Python强制要求的
- D. 必须放在文件最后一行

答案：B

解析：__name__在直接运行时为 '__main__'，被导入时为模块名，利用此特性可区分直接运行和被导入。

86. 【单选题】from package import *导入的内容由什么控制？

- A. __file__
- B. __all__
- C. __path__
- D. __doc__

答案：B

解析：__all__列表定义了from module import *时导出的公开接口。

87. 【单选题】模块搜索路径中，优先级最高的是？

- A. 标准库目录
- B. 当前目录
- C. PYTHONPATH
- D. site-packages

答案：B

解析：搜索顺序：当前目录 > PYTHONPATH > 标准库 > site-packages。

88. 【单选题】以下哪个不是合法的导入语句？

- A. import os.path
- B. from . import module
- C. import 123utils
- D. from module import func as f

答案：C

解析：模块名不能以数字开头，123utils不是合法标识符。

89. 【单选题】以下关于虚拟环境的说法，正确的是？

- A. 虚拟环境会复制Python解释器
- B. 虚拟环境用于隔离项目依赖
- C. 虚拟环境目录应提交到版本控制
- D. 全局环境比虚拟环境更安全

答案：B

解析：虚拟环境用于隔离项目依赖，避免不同项目间的依赖冲突。venv目录应加入.gitignore。

90. 【多选题】以下哪些是合法的导入方式？

- A. import module
- B. from module import name
- C. from module import *
- D. import module as alias

答案：ABCD

解析：以上都是Python支持的合法导入方式。

91. 【多选题】以下关于包管理的说法正确的是？

- A. pip是Python标准包管理器
- B. requirements.txt用于记录项目依赖
- C. setup.py用于定义包元数据
- D. pyproject.toml是现代项目的标准配置文件

答案：ABCD

解析：以上都是Python包管理的正确知识点。

92. 【多选题】以下哪些工具可以用于创建虚拟环境？

- A. venv
- B. virtualenv
- C. conda
- D. pip

答案：ABC

解析：venv是标准库工具，virtualenv和conda是第三方工具，pip是包管理器不是环境管理器。

93. 【判断题】Python 3.3之后，包目录下可以没有__init__.py文件。

答案：正确

解析：Python 3.3引入隐式命名空间包，允许无__init__.py，但显式包（regular package）仍有__init__.py更规范。

94. 【判断题】sys.path是一个列表，运行时可以通过append()添加自定义模块搜索路径。

答案：正确

解析：sys.path是模块搜索路径列表，运行时修改可动态添加路径。

95. 【判断题】使用pip install -e .可以以可编辑模式安装当前包，开发时修改代码无需重新安装。

答案：正确

解析：可编辑模式（editable install）创建指向源码的链接，修改代码立即生效。

96. 【判断题】from module import *会导入模块中的所有名称，包括以下划线开头的私有名称。

答案：错误

解析：from module import *只导入__all__列表中的名称或所有非下划线开头的名称，不会导入私有名称。

第9章 面向对象编程

97. 【单选题】self参数的作用是？

- A. 指向父类
- B. 指向类本身
- C. 指向实例对象本身
- D. 是可选的，可以省略

答案：C

解析：self指向实例对象本身，是Python的约定（可用其他名字但不推荐）。

98. 【单选题】以下关于__init__的说法，错误的是？

- A. 是构造方法，创建实例时自动调用
- B. 必须返回None
- C. 可以返回其他对象
- D. 用于初始化实例属性

答案：C

解析：__init__不能返回非None值，返回其他对象会报错。

99. 【单选题】super()函数的主要作用是？

- A. 调用父类的方法
- B. 创建父类实例
- C. 返回父类对象
- D. 检查继承关系

答案：A

解析：super()返回一个代理对象，用于调用父类的方法（遵循MRO）。

100. 【单选题】以下代码的输出是？
`class A: x = 1; class B(A): pass; class C(A): pass; B.x = 2; print(A.x, B.x, C.x)`

- A. 1 2 1
- B. 1 2 2
- C. 2 2 2
- D. 1 1 1

答案：A

解析：B.x = 2是给B创建了自己的类属性，不影响A和C的x。

101. 【单选题】@staticmethod装饰的方法有什么特点？

- A. 需要self参数
- B. 需要cls参数
- C. 既不需要self也不需要cls参数
- D. 只能访问类属性

答案：C

解析：@staticmethod与类和实例无关，不需要self或cls，类似于普通函数放在类命名空间中。

102. 【多选题】以下关于Python面向对象的说法正确的是？

- A. 类属性所有实例共享
- B. 实例属性每个实例独立
- C. __name是私有属性，完全无法访问
- D. @property可以将方法转换为属性访问

答案：ABD

解析：__name通过名称修饰变为_ClassName__name，仍可通过此名称访问，只是约定上不建议。

103. 【多选题】以下哪些是Python的特殊方法（魔术方法）？

- A. __init__
- B. __str__
- C. __eq__
- D. __call__

答案：ABCD

解析：以上都是Python的特殊方法，分别用于构造、字符串表示、相等比较、可调用对象。

104. 【多选题】以下关于继承的说法正确的是？

- A. 子类继承父类的属性和方法
- B. super()遵循MRO
- C. 子类可以重写父类方法
- D. Python不支持多重继承

答案：ABC

解析：Python支持多重继承，通过MRO（方法解析顺序）处理菱形继承问题。

105. 【判断题】Python中的私有属性__name是完全无法从外部访问的。

答案：错误

解析：__name通过名称修饰变为_ClassName__name，仍可通过_ClassName__name访问，只是约定上不建议。

106. 【判断题】@staticmethod装饰的方法既不需要self也不需要cls参数。

答案：正确

解析：@staticmethod与类和实例无关，不需要self或cls，类似于普通函数放在类命名空间中。

107.

【判断题】使用@dataclass可以自动生成__init__、__repr__、__eq__等方法，简化数据类定义。

答案：正确

解析：@dataclass（Python 3.7+）自动生成常见特殊方法，大幅减少样板代码。

108. 【判断题】抽象基类（ABC）的子类可以选择不实现抽象方法。

答案：错误

解析：继承抽象基类的子类必须实现所有抽象方法，否则无法实例化。

第10章 迭代器与生成器

109. 【单选题】以下哪个对象不是迭代器？

- A. iter([1, 2, 3])
- B. (x for x in range(5))
- C. [1, 2, 3]
- D. map(str, [1, 2, 3])

答案：C

解析：[1, 2, 3]是可迭代对象但不是迭代器，iter()才能返回迭代器。

110. 【单选题】生成器函数与普通函数的主要区别是？

- A. 生成器函数不能返回值
- B. 生成器函数使用yield而不是return来产生值
- C. 生成器函数执行更快
- D. 生成器函数只能生成数字

答案：B

解析：生成器函数使用yield暂停并返回值，状态被保存，下次从暂停处继续。

111. 【单选题】yield from的主要作用是？

- A. 终止生成器
- B. 将子生成器的值委托给外部生成器
- C. 从列表中删除元素
- D. 创建一个新的列表

答案：B

解析：yield from将子生成器的产出值直接传递给外部生成器的调用者，简化嵌套生成器。

112. 【单选题】以下代码的输出是？`def gen(): yield 1; yield 2; yield 3; print(sum(gen()))`

- A. 6
- B. 123
- C. 报错
- D. None

答案：A

解析：sum()可以接收任何可迭代对象，生成器产出1+2+3=6。

113. 【单选题】生成器表达式与列表推导式的主要区别是？

- A. 生成器表达式更快
- B. 生成器表达式惰性求值，省内存
- C. 生成器表达式可以索引访问
- D. 生成器表达式占用更多内存

答案：B

解析：生成器表达式是惰性求值，只在需要时生成值，内存占用远小于列表推导式。

114. 【多选题】以下关于迭代器的说法正确的是？

- A. 迭代器实现__iter__和__next__
- B. 迭代器只能遍历一次
- C. for循环底层使用迭代器
- D. 所有可迭代对象都是迭代器

答案：ABC

解析：可迭代对象不一定是迭代器（如列表），但迭代器一定是可迭代对象。

115. 【多选题】以下关于生成器的说法正确的是？

- A. 生成器使用yield关键字
- B. 生成器只能迭代一次
- C. 生成器表达式比列表推导式省内存
- D. send()方法可以向生成器发送值

答案：ABCD

解析：以上都是生成器的正确特性。

116. 【多选题】以下哪些是创建生成器的方式？

- A. 生成器函数（yield）

- B. 生成器表达式
- C. 列表推导式
- D. 自定义__next__方法

答案：AB

解析：生成器函数和生成器表达式是创建生成器的主要方式，列表推导式创建列表，自定义__next__创建迭代器。

117. 【判断题】生成器只能被迭代一次，第二次迭代需要重新创建生成器对象。

答案：正确

解析：生成器是单遍迭代器，迭代完后状态耗尽，需重新调用生成器函数创建新对象。

118. 【判断题】生成器表达式(x for x in range(10))比列表推导式[x for x in range(10)]占用更多内存。

答案：错误

解析：生成器表达式是惰性求值，只在需要时生成值，内存占用远小于列表推导式。

119. 【判断题】使用yield from可以替代内层for循环yield，代码更简洁且支持双向通信。

答案：正确

解析：yield from将子生成器的产出值直接传递给外部生成器的调用者，支持双向通信。

120. 【判断题】iter()函数可以将任何可迭代对象转换为迭代器。

答案：正确

解析：iter(iterable)返回一个迭代器对象，for循环底层就是调用iter()和next()。

第11章 装饰器

121. 【单选题】@decorator语法糖等价于以下哪种写法？

- A. decorator(func)
- B. func = decorator(func)
- C. func.decorator()
- D. decorator.func()

答案：B

解析：@decorator是func = decorator(func)的语法糖，装饰器接收函数并返回新函数。

122. 【单选题】以下关于functools.wraps的说法，正确的是？

- A. 用于包装函数参数
- B. 用于保留被装饰函数的元数据
- C. 用于缓存函数结果
- D. 用于限制函数调用次数

答案：B

解析：@wraps(func)将原函数的__name__、__doc__等元数据复制到包装函数上。

123. 【单选题】带参数的装饰器（如@repeat(3)）需要几层嵌套函数？

- A. 1层
- B. 2层
- C. 3层
- D. 4层

答案：C

解析：带参数装饰器需要三层：外层接收参数 -> 中间层接收函数 -> 内层包装逻辑。

124. 【单选题】类装饰器必须实现哪个特殊方法？

- A. `__init__`
- B. `__call__`
- C. `__enter__`
- D. `__iter__`

答案：B

解析：类装饰器需要实现`__call__`方法，使实例可像函数一样被调用。

125. 【单选题】以下关于lru_cache的说法，错误的是？

- A. 可以缓存函数结果
- B. `maxsize=None`表示无限制缓存
- C. 要求参数必须可哈希
- D. 适用于有副作用的函数

答案：D

解析：lru_cache不适用于有副作用的函数（如修改全局状态、写数据库），因为缓存命中时副作用不会执行。

126. 【多选题】以下关于装饰器的说法正确的是？

- A. 装饰器是接收函数并返回函数的函数
- B. 多个装饰器执行顺序是从下到上
- C. `@wraps(func)`保留原函数元数据
- D. 装饰器只能用于函数

答案：ABC

解析：装饰器可以用于类（类装饰器），也可以用于方法，Python中一切皆对象。

127. 【多选题】以下哪些是Python内置装饰器？

- A. `@property`
- B. `@classmethod`
- C. `@staticmethod`
- D. `@lru_cache`

答案：ABCD

解析：以上都是Python内置或标准库提供的装饰器。

128. 【多选题】 以下关于lru_cache的说法正确的是？

- A. 使用最近最少使用淘汰策略
- B. 可以查看缓存信息cache_info()
- C. 可以清空缓存cache_clear()
- D. 适用于所有类型的函数

答案：ABC

解析：lru_cache不适用于有副作用或参数不可哈希的函数。

129. 【判断题】 一个函数可以被多个装饰器装饰，执行顺序是从上到下。

答案：错误

解析：多个装饰器执行顺序是从下到上（靠近函数的先执行），@a @b def f()等价于f = a(b(f))。

130. 【判断题】 装饰器只能用于函数，不能用于类。

答案：错误

解析：装饰器可以用于类（类装饰器），也可以用于方法，Python中一切皆对象。

131. 【判断题】 使用@wraps(func)可以保留被装饰函数的__name__和__doc__等元数据。

答案：正确

解析：@wraps(func)将原函数的元数据复制到包装函数上，避免装饰后信息丢失。

132. 【判断题】 lru_cache(maxsize=None)表示缓存大小为0，即不缓存。

答案：错误

解析：maxsize=None表示缓存无限制，所有调用结果都会被缓存。

第12章 上下文管理器

133. 【单选题】 上下文管理器的__exit__方法接收几个参数？

- A. 0个
- B. 1个
- C. 3个
- D. 4个

答案：C

解析：__exit__(self, exc_type, exc_val, exc_tb)接收异常类型、异常值和异常追踪信息三个参数。

134. 【单选题】 如果__exit__方法返回True，会发生什么？

- A. 程序立即退出
- B. 上下文中的异常被抑制，不再传播
- C. 上下文重新执行
- D. 什么都不会发生

答案：B

解析：__exit__返回True表示异常已被处理，不再向上传播；返回False或None则继续传播。

135. 【单选题】@contextmanager装饰的生成器函数中，yield之前的代码相当于？

- A. __exit__方法
- B. __enter__方法
- C. 析构方法
- D. 构造函数

答案：B

解析：yield之前的代码在__enter__阶段执行，yield的值赋给as变量，yield之后的代码在__exit__阶段执行。

136. 【单选题】以下哪个不是contextlib提供的工具？

- A. closing
- B. suppress
- C. lru_cache
- D. redirect_stdout

答案：C

解析：lru_cache是functools模块的装饰器，不是contextlib的工具。

137. 【单选题】with语句可以同时管理多个上下文管理器，用什么分隔？

- A. 分号
- B. 逗号
- C. 冒号
- D. 竖线

答案：B

解析：with A() as a, B() as b:可以管理多个上下文，等价于嵌套的with语句。

138. 【多选题】以下关于上下文管理器的说法正确的是？

- A. 实现__enter__和__exit__方法
- B. with语句确保资源正确释放
- C. __exit__返回True抑制异常
- D. @contextmanager只能用于类

答案：ABC

解析：@contextmanager专门用于将生成器函数转换为上下文管理器，类应直接实现__enter__和__exit__。

139. 【多选题】以下哪些是contextlib提供的工具？

- A. closing
- B. suppress
- C. redirect_stdout
- D. lru_cache

答案：ABC

解析：closing、suppress、redirect_stdout都是contextlib的工具，lru_cache属于functools。

140. 【多选题】以下关于with语句的说法正确的是？

- A. 可以管理多个资源
- B. 即使发生异常也会释放资源
- C. 可以省略as部分
- D. 只能用于文件操作

答案：AB

解析：with语句可以管理任何实现了上下文管理器协议的对象，不限于文件；as部分可以省略但不推荐。

141. 【判断题】with语句可以同时管理多个上下文管理器，用逗号分隔。

答案：正确

解析：with A() as a, B() as b:可以管理多个上下文，等价于嵌套的with语句。

142. 【判断题】@contextmanager只能用于函数，不能用于类。

答案：正确

解析：@contextmanager专门用于将生成器函数转换为上下文管理器，类应直接实现__enter__和__exit__。

143. 【判断题】使用contextlib.suppress可以静默忽略指定的异常。

答案：正确

解析：with suppress(FileNotFoundError): os.remove('temp.txt')会在文件不存在时静默忽略删除错误。

144. 【判断题】__exit__方法返回False时，异常会被抑制不再传播。

答案：错误

解析：__exit__返回False或None表示异常继续传播，返回True才抑制异常。

第13章 正则表达式

145. 【单选题】re.match()和re.search()的主要区别是？

- A. match更快
- B. match从字符串开头匹配，search搜索整个字符串
- C. search只能找到第一个匹配
- D. 没有区别

答案：B

解析：re.match()必须从字符串开头匹配（等效于^），re.search()扫描整个字符串找第一个匹配。

146. 【单选题】正则表达式a+?中的?表示？

- A. 匹配0次或1次
- B. 使+变为非贪婪模式
- C. 匹配任意字符

D. 表示可选分组

答案：B

解析：量词后加?变为非贪婪模式（最小匹配），a+?匹配最少的连续a。

147. 【单选题】re.findall(r'(\d)(\d)', '1234')的结果是？

- A. ['12', '34']
- B. [('1', '2'), ('3', '4')]
- C. ['1', '2', '3', '4']
- D. ['12', '23', '34']

答案：B

解析：findall在有分组时返回元组列表，每个元组包含各分组捕获的内容。

148. 【单选题】以下哪个正则可以匹配以字母开头、后跟数字的用户名（如user123）？

- A. \d+\w+
- B. \w+\d+
- C. [a-zA-Z]\w*
- D. [a-zA-Z]\d+

答案：C

解析：[a-zA-Z]匹配字母开头，\w*匹配后续任意单词字符（包括数字和下划线）。

149. 【单选题】以下关于re.compile的说法，正确的是？

- A. 会降低匹配性能
- B. 编译后的正则对象可以被多次使用，提升重复匹配的性能
- C. 只能编译一次
- D. 不支持中文字符

答案：B

解析：re.compile()将正则编译为内部格式，避免每次匹配时重新解析，适合在循环中重复使用。

150. 【多选题】以下哪些是re模块的核心函数？

- A. match
- B. search
- C. findall
- D. sub

答案：ABCD

解析：match、search、findall、sub都是re模块的核心函数。

151. 【多选题】以下关于正则表达式的说法正确的是？

- A. .匹配任意单个字符（除换行符）
- B. ^匹配字符串开头
- C. \$匹配字符串结尾
- D. \d匹配数字[0-9]

答案：ABCD

解析：以上都是正则表达式的基本元字符含义。

152. 【多选题】以下哪些措施可以提升正则表达式性能？

- A. 使用re.compile编译复用
- B. 避免过度回溯
- C. 使用非贪婪匹配
- D. 使用原始字符串r前缀

答案：AB

解析：编译复用和避免回溯是主要性能优化手段，非贪婪匹配和r前缀与性能关系不大。

153. 【判断题】re.compile()编译后的正则对象可以被多次使用，提升重复匹配的性能。

答案：正确

解析：re.compile()将正则编译为内部格式，避免每次匹配时重新解析，适合在循环中重复使用。

154. 【判断题】在正则表达式中，.可以匹配换行符\n。

答案：错误

解析：默认情况下.不匹配换行符，需设置re.DOTALL标志才能匹配包括换行符在内的任意字符。

155. 【判断题】贪婪匹配 (.*) 匹配最长，非贪婪 (.*)? 匹配最短。

答案：正确

解析：默认量词是贪婪模式，加?后变为非贪婪模式，HTML/XML解析优先用非贪婪。

156. 【判断题】正则表达式中的原始字符串r前缀是为了避免反斜杠转义问题。

答案：正确

解析：r前缀使字符串中的反斜杠不被解释为转义字符，正则中大量使用反斜杠，使用r前缀更清晰。

第14章 多线程与多进程

157. 【单选题】Python的GIL对以下哪种任务影响最大？

- A. 文件读写
- B. 网络请求
- C. 大量数值计算
- D. 数据库查询

答案：C

解析：GIL限制CPU密集型任务的多线程并行，因为同一时刻只有一个线程执行Python字节码。

158. 【单选题】以下哪种同步机制允许同一线程多次获取锁而不会死锁？

- A. Lock
- B. RLock

- C. Semaphore
- D. Event

答案：B

解析：RLock（可重入锁）允许同一线程多次获取，内部维护获取次数计数器。

159. 【单选题】对于CPU密集型任务，推荐使用？

- A. threading.Thread
- B. multiprocessing.Process
- C. asyncio
- D. threading.Lock

答案：B

解析：CPU密集型任务应使用多进程，每个进程有独立GIL，可充分利用多核CPU。

160. 【单选题】ThreadPoolExecutor的max_workers参数表示？

- A. 最大任务数
- B. 最大线程数
- C. 最大内存使用量
- D. 最大执行时间

答案：B

解析：max_workers指定线程池中最大的工作线程数量。

161. 【单选题】以下关于多进程的说法，正确的是？

- A. 多进程间可以直接共享Python列表对象
- B. 多进程间内存独立
- C. 多进程创建开销比线程小
- D. 多进程受GIL限制

答案：B

解析：多进程间内存独立，不能直接共享对象，需通过Queue、Pipe或共享内存通信。

162. 【多选题】以下关于GIL的说法正确的是？

- A. GIL确保同一时刻只有一个线程执行Python字节码
- B. GIL对I/O密集型任务影响较小
- C. 多进程可以绕过GIL限制
- D. GIL是Python语言的特性，所有实现都有

答案：ABC

解析：GIL是CPython实现特性，Jython、IronPython等实现没有GIL。

163. 【多选题】以下哪些是线程同步机制？

- A. Lock
- B. RLock
- C. Semaphore

D. Queue

答案：ABCD

解析：Lock、RLock、Semaphore、Queue（线程安全队列）都是线程同步机制。

164. 【多选题】以下关于多线程和多进程的说法正确的是？

- A. 线程共享内存空间
- B. 进程有独立内存空间
- C. 线程适合I/O密集型任务
- D. 进程适合CPU密集型任务

答案：ABCD

解析：以上都是线程和进程的正确特性对比。

165. 【判断题】Python的多线程在I/O密集型任务中仍然可以提高效率。

答案：正确

解析：I/O操作时线程会释放GIL，其他线程可以执行，因此多线程对I/O密集型任务有效。

166. 【判断题】multiprocessing模块的进程间可以直接共享Python对象（如列表、字典）。

答案：错误

解析：多进程间内存独立，不能直接共享对象，需通过Queue、Pipe、Manager或共享内存通信。

167. 【判断题】使用Lock的with语句可以自动获取和释放锁，避免死锁。

答案：正确

解析：with lock上下文管理器自动获取和释放锁，即使发生异常也会释放锁。

168. 【判断题】在Windows上使用多进程时，必须使用if __name__ == '__main__':保护。

答案：正确

解析：Windows上多进程会重新导入主模块，缺少保护会导致子进程递归创建进程，引发崩溃。

第15章 网络编程基础

169. 【单选题】socket.AF_INET表示使用哪种地址族？

- A. IPv6
- B. IPv4
- C. Unix域套接字
- D. 蓝牙

答案：B

解析：AF_INET表示IPv4地址族，AF_INET6表示IPv6，AF_UNIX表示Unix域套接字。

170. 【单选题】socket.SOCK_STREAM对应哪种协议？

- A. UDP

- B. TCP
- C. HTTP
- D. ICMP

答案：B

解析：SOCK_STREAM表示面向连接的TCP流式套接字，SOCK_DGRAM表示UDP数据报套接字。

171. 【单选题】TCP服务端调用accept()后返回什么？

- A. 客户端IP地址
- B. 新的socket对象和客户端地址
- C. 接收到的数据
- D. 连接状态码

答案：B

解析：accept()返回一个元组(conn, address)，conn是新的socket对象用于通信，address是客户端地址。

172. 【单选题】以下哪个不是Python标准库中的HTTP客户端模块？

- A. urllib
- B. http.client
- C. requests
- D. urllib.request

答案：C

解析：requests是第三方库，不是Python标准库的一部分，需通过pip install requests安装。

173. 【单选题】以下关于UDP的说法，正确的是？

- A. UDP是面向连接的协议
- B. UDP需要三次握手
- C. UDP是无连接协议，直接发送数据
- D. UDP保证数据可靠传输

答案：C

解析：UDP是无连接协议，不需要握手建立连接，直接通过sendto()发送数据到指定地址。

174. 【多选题】以下哪些是HTTP请求方法？

- A. GET
- B. POST
- C. PUT
- D. DELETE

答案：ABCD

解析：GET、POST、PUT、DELETE都是HTTP标准方法，分别对应获取、创建、更新、删除资源。

175. 【多选题】以下关于TCP和UDP的说法正确的是？

- A. TCP面向连接，UDP无连接
- B. TCP可靠传输，UDP不保证可靠

- C. TCP是流式数据，UDP是数据报式
- D. TCP比UDP更快

答案：ABC

解析：TCP由于连接建立、确认重传等机制，通常比UDP慢，但UDP不保证可靠性。

176. 【多选题】以下哪些是处理高并发网络I/O的推荐方式？

- A. 多线程+阻塞socket
- B. select/poll/epoll多路复用
- C. asyncio异步I/O
- D. 为每个连接创建一个进程

答案：BC

解析：多路复用和asyncio是处理高并发的推荐方式，多线程/多进程在大量连接时资源开销过大。

177. 【判断题】UDP通信不需要建立连接，直接发送数据即可。

答案：正确

解析：UDP是无连接协议，不需要握手建立连接，直接通过sendto()发送数据到指定地址。

178. 【判断题】socket的recv()方法是阻塞的，直到收到数据或连接关闭。

答案：正确

解析：recv()默认是阻塞调用，程序会暂停等待数据到达；可通过setblocking(False)或settimeout()设为非阻塞。

179. 【判断题】requests库是Python标准库，无需安装即可使用。

答案：错误

解析：requests是第三方库，不是Python标准库的一部分，需通过pip install requests安装。

180. 【判断题】FastAPI框架可以自动生成OpenAPI文档，支持异步端点。

答案：正确

解析：FastAPI基于Starlette和Pydantic，自动生成交互式API文档（/docs），原生支持async/await异步端点。

第16章 元类

181. 【单选题】Python中所有类的默认元类是什么？

- A. object
- B. type
- C. class
- D. meta

答案：B

解析：type是所有类的默认元类，isinstance(MyClass, type)为True。

182. 【单选题】元类的__call__方法在什么时候被调用？

- A. 定义类时
- B. 创建类实例时
- C. 删除类时
- D. 导入模块时

答案：B

解析：当使用MyClass()创建实例时，调用的是元类的__call__，它再调用类的__new__和__init__。

183. 【单选题】type('MyClass', (Base,), {'x': 1})中第二个参数(Base,)表示？

- A. 类的属性
- B. 类的基类元组
- C. 类的元类
- D. 类的方法

答案：B

解析：type(name, bases, dict)的第二个参数是基类元组，即使只有一个基类也要用逗号（如(Base,)）。

184. 【单选题】以下哪个场景最适合使用元类？

- A. 简单的数据封装
- B. 需要统一控制多个类的创建行为（如ORM模型注册）
- C. 单次使用的工具函数
- D. 字符串处理

答案：B

解析：元类适合需要统一控制类创建行为的场景，如Django ORM的模型元类自动注册字段、管理数据库表名等。

185. 【单选题】以下关于元类和类装饰器的说法，正确的是？

- A. 元类在类创建后修改类对象
- B. 类装饰器在类创建时控制行为
- C. 类装饰器更简单，优先使用
- D. 元类只能用于创建类，不能用于控制实例化

答案：C

解析：类装饰器在类创建后修改类对象，更简单；元类在类创建时控制行为。简单场景优先使用类装饰器。

186. 【多选题】以下关于元类的说法正确的是？

- A. 元类是'类的类'
- B. 默认元类是type
- C. 自定义元类需继承type
- D. 元类可以像普通类一样被继承

答案：ABCD

解析：以上都是元类的正确特性。

187. 【多选题】 type()函数的两种用法是什么？

- A. type(obj)返回对象的类型
- B. type(name, bases, namespace)动态创建类
- C. type()返回当前模块名
- D. type()检查对象是否可调用

答案：AB

解析：type(obj)返回类型，type(name, bases, namespace)动态创建类，是type的两种核心用法。

188. 【多选题】 以下哪些场景可以考虑使用元类？

- A. 单例模式
- B. 自动注册子类
- C. 强制命名规范
- D. ORM字段自动注册

答案：ABCD

解析：以上都是元类的典型应用场景，但应优先考虑类装饰器或__init_subclass__等更简单方案。

189. 【判断题】 元类可以像普通类一样被继承，子类也会继承父类的元类行为。

答案：正确

解析：元类可以被继承，除非子类显式指定其他元类，否则会继承父类的元类。

190. 【判断题】 __new__在元类中用于控制类的实例化，__init__用于控制类的创建。

答案：错误

解析：__new__控制类的创建（分配内存），__call__控制类的实例化（Class()时调用）。

191. 【判断题】 对于简单的子类注册场景，__init_subclass__可以替代元类。

答案：正确

解析：__init_subclass__（Python 3.6+）是更简单的替代方案，适合子类注册、强制属性等简单场景。

192. 【判断题】 多重继承时可能出现metaclass conflict，需要使用type()或自定义元类解决。

答案：正确

解析：当多个父类有不同元类时，Python无法确定子类的元类，需手动解决冲突。

第17章 描述符

193. 【单选题】 以下哪个不是描述符必须实现的方法？

- A. __get__
- B. __set__
- C. __call__
- D. __delete__

答案：C

解析：描述符只需实现__get__、__set__、__delete__中的至少一个，__call__不是描述符协议方法。

194. 【单选题】数据描述符和非数据描述符的主要区别是？

- A. 数据描述符更快
- B. 数据描述符实现了__set__，优先级高于实例字典
- C. 非数据描述符可以删除属性
- D. 没有区别

答案：B

解析：数据描述符（有__set__）优先级高于实例__dict__；非数据描述符（只有__get__）优先级低于实例__dict__。

195. 【单选题】property装饰器本质上是什么？

- A. 元类
- B. 描述符
- C. 装饰器函数
- D. 生成器

答案：B

解析：property是内置的数据描述符，实现了__get__、__set__、__delete__。

196. 【单选题】当通过类访问描述符属性（如MyClass.attr）时，__get__的instance参数值是？

- A. 类的实例
- B. None
- C. 类本身
- D. 描述符对象

答案：B

解析：类访问时instance为None，owner为类本身；实例访问时instance为实例对象。

197. 【单选题】以下关于描述符的说法，正确的是？

- A. 描述符作为实例属性定义时也会生效
- B. 描述符必须作为类的属性定义才能生效
- C. 所有类属性都是描述符
- D. 描述符只能用于属性访问控制

答案：B

解析：描述符协议只在描述符作为类属性时触发，作为实例属性时Python不会调用描述符方法。

198. 【多选题】以下关于描述符的说法正确的是？

- A. 描述符实现__get__、__set__或__delete__
- B. property、classmethod、staticmethod都是描述符
- C. 数据描述符优先级高于实例__dict__
- D. 非数据描述符优先级低于实例__dict__

答案：ABCD

解析：以上都是描述符的正确特性。

199. 【多选题】以下哪些是描述符的应用场景？

- A. 类型检查
- B. 属性缓存
- C. 访问计数
- D. 范围限制

答案：ABCD

解析：描述符可用于类型检查、属性缓存、访问计数、范围限制等多种场景。

200. 【多选题】以下关于属性查找顺序的说法正确的是？

- A. 数据描述符 > 实例__dict__
- B. 实例__dict__ > 非数据描述符
- C. 类__dict__ > 实例__dict__
- D. 非数据描述符 > 类__dict__

答案：AB

解析：属性查找顺序：数据描述符 > 实例字典 > 非数据描述符 > 类字典。

201.

【判断题】描述符必须作为类的属性定义才能生效，作为实例属性定义时不会触发描述符协议。

答案：正确

解析：描述符协议只在描述符作为类属性时触发，作为实例属性时Python不会调用描述符方法。

202. 【判断题】数据描述符的优先级高于实例的__dict__，而非数据描述符的优先级低于实例的__dict__。

答案：正确

解析：属性查找顺序：数据描述符 > 实例字典 > 非数据描述符 > 类字典。

203. 【判断题】@property是Python内置的数据描述符，只能用于单个类。

答案：正确

解析：@property是内置的数据描述符，只能用于单个类。自定义描述符可复用到多个类，且可维护状态。

204. 【判断题】在描述符的__get__方法中，可以通过self.attr访问描述符自身的属性。

答案：正确

解析：self指向描述符实例，可以访问描述符自身的属性，但访问被托管属性时应使用instance.__dict__避免无限递归。

第18章 内存管理与垃圾回收

205. 【单选题】Python中对象被回收的主要机制是？

- A. 标记-清除
- B. 引用计数
- C. 分代回收
- D. 复制算法

答案：B

解析：引用计数是Python的主要回收机制，对象引用计数为0时立即回收；循环引用由gc模块补充处理。

206. 【单选题】sys.getrefcount(a)返回的引用计数通常比实际多几？

- A. 0
- B. 1
- C. 2
- D. 不确定

答案：B

解析：getrefcount()会将传入的参数临时增加一次引用，因此返回值通常比实际多1。

207. 【单选题】__slots__的主要作用是？

- A. 加速属性访问
- B. 限制实例可拥有的属性，减少内存占用
- C. 实现私有属性
- D. 自动垃圾回收

答案：B

解析：__slots__用固定数组替代动态字典存储属性，显著减少内存占用，同时禁止动态添加新属性。

208. 【单选题】weakref.ref(obj)创建的引用有什么特点？

- A. 可以阻止对象被垃圾回收
- B. 不增加引用计数，对象可被正常回收
- C. 只能用于整数
- D. 会自动复制对象

答案：B

解析：弱引用不增加引用计数，不会阻止垃圾回收器回收对象，常用于缓存和观察者模式。

209. 【单选题】以下关于循环引用的说法，正确的是？

- A. 循环引用一定会导致内存泄漏
- B. Python的gc模块可以检测并回收循环引用
- C. 循环引用只能通过gc.collect()手动回收
- D. 循环引用只发生在列表之间

答案：B

解析：Python的gc模块会自动检测和回收循环引用，不会导致内存泄漏（除非对象定义了__del__且形成循环引用）。

210. 【多选题】以下哪些是Python内存管理的机制？

- A. 引用计数
- B. 垃圾回收器（处理循环引用）
- C. 内存池（pymalloc）
- D. 对象复用（小整数缓存）

答案：ABCD

解析：以上都是Python内存管理的核心机制。

211. 【多选题】以下关于__slots__的说法正确的是？

- A. 限制实例属性数量
- B. 减少内存占用
- C. 禁止动态添加属性
- D. 继承时子类自动获得内存优化

答案：ABC

解析：子类也需定义__slots__才能获得内存优化，否则继承父类的__slots__但仍有__dict__。

212. 【多选题】以下哪些是内存优化的技巧？

- A. 使用__slots__
- B. 使用生成器替代列表
- C. 及时删除大对象并gc.collect()
- D. 使用weakref避免循环引用

答案：ABCD

解析：以上都是有效的内存优化技巧。

213. 【判断题】Python的小整数（-5到256）会被缓存复用，因此a = 5; b = 5时a is b为True。

答案：正确

解析：CPython会缓存小整数对象，因此相同小整数的is判断为True（但不应依赖此行为）。

214. 【判断题】循环引用只能通过gc.collect()手动回收，自动GC无法处理。

答案：错误

解析：Python的gc模块会自动检测和回收循环引用，gc.collect()只是手动触发，非唯一方式。

215.

【判断题】使用weakref.WeakValueDictionary可以实现自动淘汰缓存（无外部引用时自动移除）。

答案：正确

解析：WeakValueDictionary中的值是弱引用，当值对象无其他强引用时自动从字典中移除。

216. 【判断题】gc.disable()可以禁用自动垃圾回收，之后需要手动调用gc.collect()触发回收。

答案：正确

解析：gc.disable()禁用自动GC，gc.enable()恢复，gc.collect()手动触发回收。

第19章 设计模式

217. 【单选题】在Python中，以下哪种方式实现单例最简单且符合Python风格？

- A. 元类
- B. 模块级全局变量
- C. 双重检查锁
- D. 装饰器

答案：B

解析：Python模块天然就是单例（模块只导入一次），将单例对象放在模块级别是最Pythonic的方式。

218. 【单选题】策略模式在Python中通常如何简化实现？

- A. 使用继承
- B. 使用一等函数作为策略
- C. 使用全局变量
- D. 使用异常处理

答案：B

解析：Python的函数是一等公民，可直接传入作为策略，无需定义策略接口和多个策略类。

219. 【单选题】观察者模式中，Subject的主要职责是？

- A. 执行业务逻辑
- B. 维护观察者列表并通知它们
- C. 修改观察者状态
- D. 创建观察者

答案：B

解析：Subject（主题）维护观察者列表，状态变化时调用各观察者的update()方法通知它们。

220. 【单选题】适配器模式的主要目的是？

- A. 提高性能
- B. 将不兼容的接口转换为兼容的接口
- C. 创建对象
- D. 管理对象生命周期

答案：B

解析：适配器模式包装一个类，将其接口转换为客户期望的另一个接口，解决接口不兼容问题。

221. 【单选题】以下关于Python设计模式的说法，正确的是？

- A. Python的迭代器模式需要手动实现__iter__和__next__
- B. Python原生支持迭代器协议，for循环内置迭代支持
- C. 工厂模式在Python中不适用
- D. 单例模式必须使用元类实现

答案：B

解析：Python原生支持迭代器协议，for循环、yield、生成器表达式都是内置的迭代器支持。

222. 【多选题】以下哪些是设计模式的分类？

- A. 创建型
- B. 结构型
- C. 行为型
- D. 并发型

答案：ABC

解析：设计模式分为创建型（单例、工厂等）、结构型（装饰器、适配器等）、行为型（观察者、策略等）。

223. 【多选题】以下关于Python和设计模式的说法正确的是？

- A. 一等函数和鸭子类型简化了许多模式实现
- B. 装饰器模式可用Python装饰器语法直接实现
- C. 迭代器模式已内置于语言
- D. 所有Java设计模式都适用于Python

答案：ABC

解析：Python不应机械套用Java/C++的设计模式，应利用语言特性简化，遵循KISS原则。

224. 【多选题】以下哪些是创建型设计模式？

- A. 单例
- B. 工厂
- C. 建造者
- D. 观察者

答案：ABC

解析：单例、工厂、建造者都是创建型模式，观察者是行为型模式。

225.

【判断题】Python的迭代器模式需要手动实现__iter__和__next__，因为语言本身不提供迭代支持。

答案：错误

解析：Python原生支持迭代器协议，for循环、yield、生成器表达式都是内置的迭代器支持。

226. 【判断题】工厂模式将对象的创建逻辑集中管理，便于后续扩展和维护。

答案：正确

解析：工厂模式将对象创建集中到一个地方，新增产品只需修改工厂，无需改动使用方代码。

227. 【判断题】在Python中，模块天然单例，因此将单例对象放在模块级别是最简单的方式。

答案：正确

解析：模块只导入一次，模块级全局变量天然单例，是最Pythonic的实现方式。

228. 【判断题】策略模式在Python中应使用策略类实现，不能使用函数。

答案：错误

解析：Python的函数是一等公民，可直接传入作为策略，无需定义策略接口和多个策略类，代码更简洁。

第20章 性能优化

229. 【单选题】以下哪种方式测量小段代码性能最精确？

- A. `time.time()`
- B. `timeit.timeit()`
- C. `datetime.now()`
- D. `os.times()`

答案：B

解析：timeit自动禁用垃圾回收、重复执行取平均，消除系统噪声，适合测量小段代码。

230. 【单选题】`functools.lru_cache`的`maxsize=None`表示？

- A. 缓存大小为0
- B. 缓存无限制
- C. 缓存大小为1
- D. 禁用缓存

答案：B

解析：`maxsize=None`表示缓存无大小限制，所有调用结果都会被缓存。

231. 【单选题】为什么`result.append`绑定到局部变量后循环更快？

- A. 局部变量访问比属性查找快
- B. `append`方法被加速了
- C. 列表变小了
- D. 没有区别

答案：A

解析：局部变量访问是数组索引（FAST），属性查找需要字典查找（LOAD_ATTR），局部变量绑定避免了每次循环的属性查找开销。

232. 【单选题】大量字符串拼接时，推荐的方式是？

- A. 使用+循环拼接
- B. 使用`join()`方法
- C. 使用f-string循环拼接
- D. 使用%格式化

答案：B

解析：`join()`预先分配足够内存，只创建一次新字符串；+拼接会创建大量中间字符串对象。

233. 【单选题】以下关于性能优化的说法，正确的是？

- A. 应优先优化所有代码
- B. 过早优化是万恶之源，先确保正确再优化瓶颈

- C. 列表推导式总是比for循环慢
- D. cProfile适合生产环境长期运行

答案：B

解析：先确保代码正确，再用cProfile找到真正的热点（80/20法则），不要凭感觉优化。

234. 【多选题】以下哪些是Python性能分析工具？

- A. timeit
- B. cProfile
- C. line_profiler
- D. memory_profiler

答案：ABCD

解析：timeit测量小段代码，cProfile标准库分析器，line_profiler逐行分析，memory_profiler内存分析。

235. 【多选题】以下哪些是常见的性能优化策略？

- A. 使用lru_cache避免重复计算
- B. 使用列表推导式替代for循环
- C. 使用join()替代+拼接字符串
- D. 使用生成器替代列表

答案：ABCD

解析：以上都是有效的性能优化策略。

236. 【多选题】以下关于NumPy的说法正确的是？

- A. NumPy向量化操作比Python循环快
- B. NumPy数组比Python列表更紧凑
- C. NumPy适合所有类型的数据处理
- D. NumPy用C实现数值计算

答案：ABD

解析：NumPy适合数值计算，不是所有数据处理场景都适用（如字符串处理、复杂对象）。

237. 【判断题】cProfile会显著增加程序运行时间，因此不适合在生产环境使用。

答案：正确

解析：cProfile是纯Python分析器，开销较大，通常在开发和测试环境使用；生产环境可用yappi或py-spy。

238. 【判断题】列表推导式通常比等价的for循环更快，因为推导式在C层面执行。

答案：正确

解析：列表推导式在字节码层面优化更好，且CPython对其有专门优化，通常比手动for循环快10-30%。

239. 【判断题】局部变量访问比全局变量快，因为局部变量是数组索引，全局变量需要字典查找。

答案：正确

解析：局部变量访问是LOAD_FAST（数组索引），全局变量是LOAD_GLOBAL（字典查找），局部变量绑定可优化性能。

240. 【判断题】NumPy向量化操作比Python循环快100倍，因此所有循环都应替换为NumPy操作。

答案：错误

解析：NumPy适合数值计算，但非数值场景（如字符串处理、复杂逻辑）不适合，且NumPy数组创建也有开销。

第21章 异步编程

241. 【单选题】`async def`定义的函数被调用时返回什么？

- A. 立即执行函数体
- B. 一个协程对象
- C. None
- D. 一个线程

答案：B

解析：`async def`函数调用时返回协程对象（coroutine object），不会立即执行，需要通过`await`或`asyncio.run()`驱动。

242. 【单选题】`await`关键字的作用是什么？

- A. 阻塞当前线程直到操作完成
- B. 挂起当前协程，让出控制权给其他协程
- C. 创建新线程
- D. 终止程序

答案：B

解析：`await`挂起当前协程，将控制权交还事件循环，让其他协程有机会执行，不会阻塞线程。

243. 【单选题】`asyncio.gather()`的作用是？

- A. 顺序执行多个协程
- B. 并发运行多个可等待对象并收集结果
- C. 取消所有任务
- D. 创建线程池

答案：B

解析：`asyncio.gather(*aws)`并发运行所有传入的可等待对象，等待全部完成后返回结果列表。

244. 【单选题】以下哪个不能在协程外部直接使用？

- A. `asyncio.run()`
- B. `await`
- C. `asyncio.sleep()`
- D. `asyncio.create_task()`

答案：B

解析：`await`只能在`async def`函数内部使用，在普通函数或全局作用域使用会报`SyntaxError`。

245. 【单选题】 以下关于asyncio的说法，正确的是？

- A. asyncio使用多线程实现并发
- B. asyncio是单线程事件循环
- C. asyncio可以绕过GIL限制
- D. asyncio适合CPU密集型任务

答案：B

解析：asyncio是单线程事件循环，利用协程切换实现并发，不涉及多线程，因此GIL不是限制因素（但也没有绕过GIL）。

246. 【多选题】 以下关于异步编程的说法正确的是？

- A. async def定义协程函数
- B. await挂起当前协程
- C. asyncio.run()运行入口协程
- D. create_task()将协程包装为Task

答案：ABCD

解析：以上都是asyncio的核心概念和正确用法。

247. 【多选题】 以下哪些是异步编程的适用场景？

- A. 大量并发网络连接
- B. 高并发Web服务器
- C. CPU密集型计算
- D. 大量文件I/O操作

答案：ABD

解析：asyncio适合I/O密集型场景，CPU密集型任务应使用多进程。

248. 【多选题】 以下关于await asyncio.sleep(1)的说法正确的是？

- A. 会阻塞整个程序1秒
- B. 是非阻塞的
- C. 协程挂起期间事件循环可以调度其他协程
- D. 等同于time.sleep(1)

答案：BC

解析：await asyncio.sleep(1)是非阻塞的，协程挂起期间事件循环可以调度其他协程执行。

249. 【判断题】 asyncio使用多线程实现并发，因此可以绕过GIL限制。

答案：错误

解析：asyncio是单线程事件循环，利用协程切换实现并发，不涉及多线程，因此GIL不是限制因素（但也没有绕过GIL）。

250. 【判断题】 await asyncio.sleep(1)会阻塞整个程序1秒钟。

答案：错误

解析：await asyncio.sleep(1)是非阻塞的，协程挂起1秒期间事件循环可以调度其他协程执行。

251. 【判断题】在协程中调用阻塞I/O（如requests.get、time.sleep）会阻塞整个事件循环。

答案：正确

解析：协程中应避免阻塞调用，使用aiohttp替代requests，使用asyncio.sleep替代time.sleep。

252. 【判断题】asyncio.gather返回结果按传入顺序排列，与完成顺序无关。

答案：正确

解析：asyncio.gather返回按传入顺序排列的结果列表，即使协程完成顺序不同。

第22章 类型提示与静态检查

253. 【单选题】Python的类型提示在运行时有什么作用？

- A. 强制类型检查，类型不匹配会报错
- B. 没有任何运行时作用，仅用于静态检查
- C. 自动转换类型
- D. 提高运行速度

答案：B

解析：Python的类型提示是可选的、运行时被忽略的注释信息，仅用于静态类型检查工具（如mypy）和IDE提示。

254. 【单选题】Optional[str]等价于以下哪种写法？

- A. str | None (Python 3.10+)
- B. Union[str]
- C. str or None
- D. Any

答案：A

解析：Python 3.10引入X | Y语法，str | None等价于Optional[str]和Union[str, None]。

255. 【单选题】Protocol的主要用途是？

- A. 定义抽象基类
- B. 实现结构子类型（鸭子类型）检查
- C. 网络通信协议
- D. 替代继承

答案：B

解析：Protocol定义接口规范，只要类实现了协议中的方法（无论是否显式继承），静态检查就认为是该协议的子类型。

256. 【单选题】Callable[[int, str], bool]表示？

- A. 返回bool的列表
- B. 接收int和str两个参数、返回bool的函数
- C. 一个布尔值

D. 一个类

答案：B

解析：Callable[[参数类型列表], 返回类型]，[int, str]表示接收int和str两个参数。

257. 【单选题】以下关于mypy的说法，正确的是？

- A. mypy是Python官方运行时类型检查器
- B. mypy是静态类型检查器，在代码运行前分析类型
- C. mypy会在程序运行时检查类型
- D. mypy可以自动修复类型错误

答案：B

解析：mypy是静态类型检查器，在代码运行前分析类型，不会在运行时介入。

258. 【多选题】以下哪些是Python类型提示的常用类型？

- A. List[int]
- B. Dict[str, int]
- C. Optional[str]
- D. Union[int, str]

答案：ABCD

解析：以上都是typing模块提供的常用类型注解。

259. 【多选题】以下关于类型提示的说法正确的是？

- A. 类型提示在运行时被忽略
- B. 可以提高代码可读性
- C. IDE可以利用类型提示提供智能补全
- D. mypy可以检查类型错误

答案：ABCD

解析：以上都是类型提示的正确特性和优势。

260. 【多选题】以下哪些是Python 3.9+的类型提示新特性？

- A. 内置泛型list[int]
- B. X | Y替代Union[X, Y]
- C. X | None替代Optional[X]
- D. 类型提示在运行时强制执行

答案：ABC

解析：Python 3.9+支持内置泛型，3.10+支持X | Y语法，但类型提示始终不会在运行时强制执行。

261. 【判断题】Python 3.9起可以直接使用list[int]而无需从typing导入List。

答案：正确

解析：Python 3.9起内置容器类型支持泛型语法（list[int]、dict[str, int]），不再需要typing.List、typing.Dict。

262. 【判断题】mypy是Python官方运行时类型检查器，会在程序运行时检查类型。

答案：错误

解析：mypy是静态类型检查器，在代码运行前分析类型，不会在运行时介入。

263. 【判断题】使用from __future__ import annotations可以避免运行时导入typing类型。

答案：正确

解析：from __future__ import annotations (Python 3.7+) 将类型提示存储为字符串，避免运行时导入开销。

264. 【判断题】Protocol定义接口规范，只要类实现了协议中的方法，静态检查就认为是该协议的子类型。

答案：正确

解析：Protocol实现结构子类型（鸭子类型）检查，无需显式继承，只要实现所需方法即可。

第23章 C扩展开发

265. 【单选题】ctypes的主要用途是？

- A. 编写C代码
- B. 在Python中调用动态链接库（.so/.dll）
- C. 替代Python解释器
- D. 调试C程序

答案：B

解析：ctypes是Python标准库，用于加载C动态链接库并调用其中的函数，无需编写C扩展代码。

266. 【单选题】Cython文件的后缀名是什么？

- A. .c
- B. .cpp
- C. .pyx
- D. .pyd

答案：C

解析：Cython源文件后缀为.pyx，编译后生成C代码再编译为.so（Linux）或.pyd（Windows）。

267. 【单选题】使用ctypes调用C函数前，为什么要设置argtypes和restype？

- A. 提高运行速度
- B. 让Python正确转换参数和返回值类型
- C. 检查C代码语法
- D. 没有特别作用

答案：B

解析：argtypes和restype告诉ctypes如何正确转换Python类型到C类型，以及返回值如何转回Python类型。

268. 【单选题】以下哪种方式不需要编写C/C++代码就能调用C库？

- A. Python C API
- B. ctypes
- C. Cython
- D. pybind11

答案：B

解析：ctypes直接调用已编译好的动态库，无需编写C/C++代码；其他方式都需要编写C/C++扩展代码。

269. 【单选题】以下关于C扩展的说法，正确的是？

- A. C扩展不能绕过GIL
- B. C扩展可以绕过GIL实现真正并行
- C. ctypes只能调用C标准库
- D. Cython不需要编译即可使用

答案：B

解析：C扩展在执行C代码时可以释放GIL（Py_BEGIN_ALLOW_THREADS / Py_END_ALLOW_THREADS），实现真正的并行计算。

270. 【多选题】以下哪些是Python C扩展的主要方式？

- A. Python C API
- B. ctypes
- C. Cython
- D. pybind11

答案：ABCD

解析：以上都是Python调用C/C++代码的主要方式，各有适用场景。

271. 【多选题】以下关于ctypes的说法正确的是？

- A. 可以调用动态链接库
- B. 需要设置argtypes和restype
- C. 只能调用C标准库
- D. 不需要编译C代码

答案：ABD

解析：ctypes可以调用任何动态链接库（包括自定义编译的.so、.dll），不限于C标准库。

272. 【多选题】以下关于Cython的说法正确的是？

- A. Cython是Python的超集
- B. 使用cdef声明C类型变量
- C. 编译后生成C代码
- D. 无需编译即可运行.pyx文件

答案：ABC

解析：Cython需要编译.pyx文件为C扩展后才能使用，不能直接使用。

273. 【判断题】C扩展可以绕过Python的GIL，实现真正的多线程并行。

答案：正确

解析：C扩展在执行C代码时可以释放GIL（Py_BEGIN_ALLOW_THREADS / Py_END_ALLOW_THREADS），实现真正的并行计算。

274. 【判断题】ctypes只能调用C标准库，不能调用自定义编译的动态库。

答案：错误

解析：ctypes可以加载任何动态链接库（包括自定义编译的.so、.dll、.dylib），不限于C标准库。

275. 【判断题】Cython中cdef声明的C类型变量在编译为C后直接使用C运算，大幅提升循环性能。

答案：正确

解析：Cython中cdef声明的C类型变量在编译为C后直接使用C运算，避免Python对象开销，大幅提升性能。

276. 【判断题】pybind11是一个C++库，可以简化C++扩展的编写，支持现代C++特性。

答案：正确

解析：pybind11是轻量级的C++库，自动处理类型转换、异常传播等，比Python C API更易用。